

PROJECT IMPLEMENTATION DOCUMENT

Selenium GUI Automation Framework

testautomationpractice.blogspot.com

Document	DOC-01
Trainee	Suriseti Venkata Kishore
Trainer	Chandan Navin
Organization	Wipro Limited
Program	SDET Training
Date	June 2026
GitHub	https://github.com/VenkataKishore123/Capstone_Project.git

1. Project Overview

This document describes the complete implementation of the Selenium GUI Automation Framework developed as the Capstone Project for the Wipro SDET training program. The project automates all interactive GUI elements on testautomationpractice.blogspot.com using Java, Selenium WebDriver, TestNG, Maven, ExtentReports, and Log4j2.

2. Project Details

Attribute	Details
Project Name	Selenium GUI Automation Framework
Project Type	SDET Capstone Project
Application URL	https://testautomationpractice.blogspot.com
Language	Java 11
Framework	Selenium WebDriver 4.44.0
Test Framework	TestNG 7.9.0
Build Tool	Apache Maven 3.x
Browser Management	WebDriverManager 5.7.0
Reporting	ExtentReports 5.1.1
Logging	Log4j2 2.23.1
IDE	Eclipse IDE
Design Pattern	Page Object Model (POM)
Browser	Google Chrome 149.0.7827.54
OS	Windows 11

3. Project Folder Structure

The project follows a standard Maven directory layout with clear separation of concerns:

File / Folder	Purpose
pom.xml	Maven build file – all dependencies and Surefire plugin configuration
src/test/java/com/automation/tests/BaseTest.java	@BeforeSuite launches Chrome and opens URL. @AfterSuite closes browser and flushes report.
src/test/java/com/automation/tests/GuiElementsTest.java	Contains 34 @Test methods. Each test creates ExtentTest node, calls POM

	methods, asserts results, and captures screenshots.
src/test/java/com/automation/pages/GuiElementsPage.java	Page Object Model class. All locators and interaction methods centralised here.
src/test/java/com/automation/utils/ExtentReportManager.java	Singleton class managing the single ExtentReports instance for the entire test run.
src/test/java/com/automation/utils/ScreenshotUtil.java	Captures full-page PNG screenshots. Auto-creates screenshots/ folder at runtime.
src/test/resources/testng.xml	TestNG suite definition. Maven Surefire plugin reads this file to execute tests.
src/test/resources/log4j2.xml	Log4j2 configuration. Suppresses Selenium noise and keeps INFO level for test output.
screenshots/	Auto-created at runtime. Stores all before and after PNG screenshots.
test-output/ExtentReport.html	Rich HTML report generated after every test run.

4. Framework Architecture

4.1 Design Pattern — Page Object Model

The framework uses the Page Object Model (POM) design pattern. All web element locators and interaction methods are centralised in GuiElementsPage.java. Test methods in GuiElementsTest.java call page methods only — they never call driver.findElement() directly.

Benefits of POM in this project:

- Single location to update locators when website changes
- Test code is clean and readable
- Same page methods reused across multiple test cases
- Easy to scale — add new page classes for new pages

4.2 Layer Separation

Layer	Class	Responsibility
Test Layer	GuiElementsTest.java	18 @Test methods with assertions and screenshots
Page Layer	GuiElementsPage.java	
Base Layer	BaseTest.java	Browser lifecycle management

Utility Layer	ExtentReportManager.java	HTML report management
Utility Layer	ScreenshotUtil.java	PNG screenshot capture and saving

5. Implementation Steps

The framework was built in the following order:

1. Created Maven project in Eclipse with GroupId: com.automation, ArtifactId: selenium-gui-automation.
2. Created 3 packages: tests, pages, utils.
3. Added all 7 dependencies in pom.xml including Selenium, TestNG, WebDriverManager, ExtentReports, Log4j2.
4. Wrote ExtentReportManager.java using Singleton pattern for centralised report management.
5. Wrote ScreenshotUtil.java with TakesScreenshot interface for PNG capture
6. Wrote BaseTest.java with @BeforeSuite and @AfterSuite for browser lifecycle
7. Wrote GuiElementsPage.java with 30+ locators and action methods following POM
8. Wrote GuiElementsTest.java with 34 @Test methods covering all GUI elements
9. Added testng.xml suite definition and log4j2.xml logging configuration
10. Ran Maven clean and install to validate and execute all tests

6. TestNG Execution Flow

Order	Annotation	Action
1st	@BeforeSuite	Chrome launched, URL opened, ExtentReport initialised
2nd	@BeforeClass	GuiElementsPage object created with driver and wait
3rd-20th	@Test (x34)	All 34 test cases executed in priority order
Last	@AfterSuite	Browser closed, ExtentReport flushed to disk

7. Selenium Concepts Used

Concept	Usage in Project
WebDriver	Launches and controls Chrome browser throughout the test run
By Locators	
Select Class	Handles Country, Color and Animals dropdowns via selectByVisibleText()

Actions Class	moveToElement, doubleClick, dragAndDrop, dragAndDropBy for advanced interactions
JavascriptExecutor	Scrolls page up and down, sets read-only date field values
Alert Handling	switchTo().alert() for Simple, Confirm and Prompt alert types
Implicit Wait	10 seconds set globally in BaseTest for all element lookups
Explicit Wait	WebDriverWait with ExpectedConditions.alertIsPresent() for alerts
TakesScreenshot	Selenium interface used in ScreenshotUtil for full-page PNG capture
WebDriverManager	Auto-downloads matching ChromeDriver binary. No manual setup needed.

8. Test Execution Results

Metric	Result
Total Test Cases	34
Tests Passed	34
Tests Failed	0 (after bug fixes)
Tests Skipped	0
Total Execution Time	~49 seconds
Screenshots Captured	20+ PNG files
Report Generated	test-output/ExtentReport.html
Bugs Found	3 (3 Fixed, — no functional impact)

9. How to Run

Option 1 — Run via TestNG in Eclipse

1. Right-click testng.xml in src/test/resources
2. Select Run As → TestNG Suite
3. View results in TestNG tab at bottom of Eclipse
4. Open test-output/ExtentReport.html in browser for full report

Option 2 — Run via Maven

1. Right-click project → Run As → Maven clean
2. Right-click project → Run As → Maven install
3. Open test-output/ExtentReport.html in browser for full report